

Introduction to Memory Allocation in 'C'

Memory allocations, in general, mean where computer programs and services are executed to reserve partially or complete space or virtual memory of a computer, this process is known as memory allocation.

This process is hardware operation and is achieved by memory management through Operating systems and software applications. In general, there are static and dynamic memory allocations, whereas, in C programming language, we will see about dynamic memory allocation where programs are allocated during run time in memory and static memory allocation is a process of allocating memory while writing the C program which means memory is allocated at compile time.





Static Memory

Allocated at compile time



Dynamic Memory

Allocated at runtime



Hardware Operation

Managed by OS and applications



Static Memory Allocation in 'C'

As we discussed static memory allocation is the allocation of memory for the data variables when the computer programs start. This type of allocation is applied to only global variables, file scope variables and also to those variables that are declared as static.

This type of allocation is having a drawback when you are allocating memory we should know the exact memory before allocating as this process allocates fixed memory and cannot be changed after allocating.



Features of Static Memory Allocation

- This type of allocation allocates variables permanently
- The memory cannot be reused and is, therefore, less efficient
- This allocation uses the stack for implementing the allocation process



Example:

```
#include <stdio.h>

void play
{
    int x;
}

int main()
{
    int y;
```

```
int c[10];  
return 1;  
}
```

2/4

Static Variables & Dynamic Memory

A variable can internally or externally be declared as static in which its value persists until the end of the program, where this can be done using the keyword static before the variable declaration. There can be internal or external static variables that are declared inside or outside the function.



Static Variable Example:

```
#include<stdio.h>  
void stat(void);  
  
int main()
```

```
{
    int i;
    for(i=1; i<=3 ; i++)
        stat();
    return 1;
}

void stat(void)
{
    static int n = 0;
    n = n+1;
    printf("n = %d""\n", n);
}
```

Output:

```
n = 1
n = 2
n = 3
```



Dynamic Memory Allocation in 'C'

As discussed above dynamic memory allocation is allocation of memory during runtime or during program execution. Dynamic memory allocation provides different functions in the C programming language.



Stack

In this section, local variable or automatic variable and information regarding the address of function call are stored such as stack pointer.



Heap

This is the part of memory where dynamic memory allocation take place. Now for dynamic memory allocation following standard library functions are essential that are defined in the standard library.



Difference Static & Dynamic Memory Allocation

Static Memory Allocation

Memory is allocated at compile time.

Dynamic Memory Allocation

Memory is allocated at run time.

Memory cannot be increased while executing program.

Memory can be increased while executing program.

Used in array.

Used in linked list.



Methods used for dynamic memory allocation:



malloc()

Allocates single block of requested memory.



calloc()

Allocates multiple block of requested memory.



realloc()

Reallocates the memory occupied by malloc() or calloc() function.



free()

Frees the dynamically allocated memory.

💡 Key Takeaway

Understanding the difference between static and dynamic memory allocation is crucial for efficient memory management in C programming. Static allocation provides fixed memory at compile time, while dynamic allocation offers flexibility during runtime.

